

Object-Oriented Programming in .NET

Interview Questions & Correct Answers

Prepared from our interactive OOP practice session

این فایل شامل تمام سؤال‌هایی است که در همین جلسه پرسیده شد، همراه با جواب صحیح و مناسب مصاحبه. پاسخ‌ها عمداً کوتاه و قابل گفتن در مصاحبه نوشته شده‌اند.

1. Classes, Objects, State, and Encapsulation

1. Class vs Object

Question: What is the difference between a class and an object?

Interview answer: A class defines a type and describes its data and behavior. An object is a runtime instance of that class. Different objects created from the same class can hold different state.

```
public class Payment
{
    public decimal Amount { get; set; }
}

var p1 = new Payment();
var p2 = new Payment();
```

نکته: کلاس تعریف نوع است و آبجکت نمونه‌ای واقعی از آن نوع در زمان اجراست.

2. Field vs Property

Question: What is the difference between a field and a property, and why do we usually avoid public fields?

Interview answer: A field stores data directly. A property provides controlled access to data through get and set accessors. Properties support encapsulation because we can restrict or validate how the state is read or changed.

را می‌دهد state نکته: فیلد مستقیماً داده را نگه می‌دارد؛ پراپرتی امکان کنترل دسترسی و تغییر

3. Constructor and Valid State

Question: What is a constructor, and how can it help create an object in a valid state?

Interview answer: A constructor is a special member of a class with the same name as the class and no return type. We use it to initialize an object. It can receive parameters, validate them, and ensure that the object starts in a valid state.

```
public Payment(decimal amount, string referenceNumber)
{
    if (amount <= 0)
        throw new ArgumentOutOfRangeException(nameof(amount));

    if (string.IsNullOrEmpty(referenceNumber))
        throw new ArgumentException("Reference number is required.");

    Amount = amount;
    ReferenceNumber = referenceNumber;
}
```

کند enforce نکته: کانستراکتور می‌تواند از همان لحظه ساخت، قوانین اولیه را

4. Encapsulation with BankAccount

Question: How would you redesign BankAccount so Balance cannot become negative?

Interview answer: Keep the setter private and expose methods such as Deposit and Withdraw. Those methods validate the requested operation before changing Balance. This keeps the object in a valid state and prevents external code from bypassing business rules.

```
public class BankAccount
{
    public decimal Balance { get; private set; }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(nameof(amount));
        Balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(nameof(amount));
        if (amount > Balance)
            throw new InvalidOperationException("Insufficient balance.");
        Balance -= amount;
    }
}
```

از مسیر متدهای معنی‌دار دامنه انجام می‌شود state مستقیم، تغییر set نکته: به‌جای

5. Why private set?

Question: Why use public decimal Balance { get; private set; } instead of a public setter?

Interview answer: Because the class should be responsible for protecting its own state. A private setter prevents outside code from changing Balance directly. Operations such as Withdraw can enforce business rules first and keep the object in a valid state.

است responsibility + business rule + encapsulation نکته: این همان ترکیب

6. Who Owns the Business Rule?

Question: Should checking Balance >= amount be the responsibility of the service or the BankAccount domain object?

Interview answer: The BankAccount domain object should own that business rule because it is responsible for protecting its own state. With encapsulation, the rule stays close to the data it protects, so callers cannot accidentally bypass it.

باشد object را حفظ می‌کند بهتر است داخل خود آن domain object یک invariant نکته: قانونی که مستقیماً

2. Inheritance and Polymorphism

7. Inheritance

Question: What is inheritance, and what does CardPayment inherit from Payment?

Interview answer: Inheritance models an is-a relationship. A derived class inherits accessible members from its base class, such as public and protected members. For example, CardPayment is a Payment and can inherit members such as Amount.

معنی‌دار باشد is-a باید رابطه inheritance نکته: برای

8. Access Modifiers

Question: What is the difference between private, protected, and public?

Interview answer: private members are accessible only inside the declaring class. protected members are accessible inside the declaring class and derived classes. public members are accessible from any code that can access the type.

معمولاً برای اعضای است که باید برای کلاس‌های مشتق قابل استفاده باشند: protected نکته:

9. Runtime Polymorphism

Question: What will be printed by `Payment payment = new CardPayment(); payment.Pay();` and why?

Interview answer: If Pay is virtual in Payment and overridden in CardPayment, Card Payment will be printed. The reference type is Payment, but the runtime object is CardPayment, so the overridden implementation is selected at runtime. This is runtime polymorphism.

```
public class Payment
{
    public virtual void Pay() => Console.WriteLine("Payment");
}

public class CardPayment : Payment
{
    public override void Pay() => Console.WriteLine("Card Payment");
}

Payment payment = new CardPayment();
payment.Pay(); // Card Payment
```

می‌توانند متفاوت باشند runtime و نوع واقعی reference نکته: نوع

10. Overloading vs Overriding

Question: What is the difference between overloading and overriding?

Interview answer: Overloading means multiple methods have the same name but different parameter signatures. Overriding means a derived class provides a new implementation for an inherited virtual or abstract member with the same signature.

```
void Pay(int amount) { }
void Pay(decimal amount) { } // overloading

public override void Pay() { } // overriding
```

است runtime polymorphism پایهی overriding انتخاب می‌شود؛ compile-time معمولاً Overloading نکته:

3. Abstraction, Interfaces, and Composition

11. Abstraction

Question: What is abstraction, and why use an abstract Payment class?

Interview answer: Abstraction means exposing essential behavior while hiding unnecessary implementation details. An abstract class cannot be instantiated directly. It can contain abstract members and shared implemented members. We can use an abstract Payment when only concrete payment types should be created.

است C# یکی از ابزارهای پیاده‌سازی آن در abstract class یک مفهوم طراحی است؛ Abstraction نکته:

12. Interface vs Abstract Class

Question: What is the difference between an interface and an abstract class?

Interview answer: An interface mainly defines a contract and a class can implement multiple interfaces. An abstract class is a base class that can provide shared implementation and hold state. A class can inherit from only one class.

می‌تواند مناسب باشد abstract class shared state/behavior مناسب‌تر است؛ برای interface چندگانه معمولاً contract نکته: برای

13. When to Choose Interface

Question: When would you choose an interface instead of an abstract class?

Interview answer: Use an interface when you mainly need a contract, multiple unrelated implementations, or flexible substitution. Use an abstract class when closely related types need shared behavior or shared state.

مشخص hierarchy می‌بندد، نه به یک contract وابستگی را به قابلیت یا interface نکته:

14. Composition vs Inheritance

Question: For Car and Engine, should Car inherit Engine or contain Engine?

Interview answer: Car should contain Engine because Car has an Engine; it is not an Engine. Use inheritance for an is-a relationship and composition for a has-a relationship.

نکته: is-a → inheritance و has-a → composition.

15. Why Composition is Often Safer

Question: For OrderService and EmailService, is inheritance or composition better?

Interview answer: Composition is better. OrderService should depend on an IEmailService rather than inherit from EmailService. This reduces coupling, improves flexibility, and allows the dependency to be replaced or mocked easily.

```
public class OrderService
{
    private readonly IEmailService _emailService;

    public OrderService(IEmailService emailService)
    {
        _emailService = emailService;
    }
}
```

```
}
```

به دست می‌آید dependency on abstraction با composition در این مثال بیشتر از ترکیب coupling نکته: کاهش

4. Dependency Injection in .NET

16. Dependency Injection

Question: What is Dependency Injection, and why is it useful in .NET applications?

Interview answer: Dependency Injection means providing a class with its dependencies from outside instead of creating them inside the class. In .NET, the DI container can resolve and inject dependencies, commonly through constructors. This supports loose coupling and improves testability.

از بیرون تزریق می‌شود dependency کند؛ new هایش را dependency نکته: کلاس نباید بی‌دلیل خودش

17. DI Lifetimes

Question: What is the difference between Transient, Scoped, and Singleton?

Interview answer: Transient creates a new instance each time the service is resolved. Scoped creates one instance per scope; in ASP.NET Core an HTTP request usually represents one scope. Singleton uses one instance for the entire application lifetime.

است ASP.NET Core در scope فقط رایج‌ترین نمونه‌ی HTTP request؛ per-scope یعنی Scoped: نکته

18. Scoped inside Singleton

Question: What is wrong with injecting a Scoped service into a Singleton service?

Interview answer: A Singleton should not directly capture a Scoped dependency because the shorter-lived dependency can effectively live as long as the Singleton. This is called a captive dependency. Usually the lifetimes should be redesigned; when appropriate, IServiceScopeFactory can create an explicit scope.

```
using var scope = _scopeFactory.CreateScope();  
var dbContext = scope.ServiceProvider.GetRequiredService<MyDbContext>();
```

آن را تشخیص می‌دهد validation یا هنگام runtime در DI container نیست؛ معمولاً compile-time نکته: این خطا

5. Encapsulation vs Abstraction

19. Encapsulation vs Abstraction

Question: What is the difference between Encapsulation and Abstraction?

Interview answer: Encapsulation is about protecting and controlling an object's internal state and implementation. Abstraction is about exposing only the essential operations while hiding unnecessary complexity. In short: encapsulation protects the inside; abstraction simplifies what the outside needs to see.

```
public interface IPaymentGateway  
{  
    Task PayAsync(decimal amount);  
}
```

```
// Caller knows what operation exists, not the internal implementation.
```

نکته: برای حفظ تفاوت Encapsulation → protect state؛ Abstraction → hide unnecessary complexity.

Quick Interview Review

Concept	Short interview definition
Class	Defines a type
Object	Runtime instance of a class
Encapsulation	Protect and control internal state
Abstraction	Expose essentials, hide unnecessary complexity
Inheritance	is-a relationship
Composition	has-a relationship
Overloading	Same method name, different parameters
Overriding	Derived class replaces virtual/abstract behavior
Interface	Contract; multiple interfaces can be implemented
Abstract class	Base class with shared behavior/state
Transient	New instance per resolution
Scoped	One instance per scope
Singleton	One instance for application lifetime

پیشنهاد مرور: هر روز ۵ سؤال را بدون نگاه کردن به جواب ها با صدای بلند پاسخ بده. هدف، جواب های کوتاه و طبیعی برای مصاحبه است، نه حفظ کردن متن کلمه به کلمه.